code 1.cpp | code 2.cpp | code 3.cpp

```cpp
#include <iostream>
using namespace std;

class Stack
{
private:
    struct Node
    {
        int data;
        Node* next;
    };
    Node* topNode;
    int count;
public:

    Stack()
    {
        topNode = NULL;
        count = 0;
    }

    ~Stack()
    {
        while (topNode != NULL)
        {
            Node* temp = topNode;
            topNode = topNode->next;
            delete temp;
        }
    }

    bool isEmpty()
    {
        return topNode == NULL;
    }

    void push(int value)
    {
```

code 1.cpp | code 2.cpp | code 3.cpp

```cpp
void push(int value)
{
    Node* newNode = new Node;

    newNode->data = value;
    newNode->next = topNode;

    topNode = newNode;
    count++;
}

int pop()
{
    if (isEmpty())
    {
        cout << "Stack Underflow!" << endl;
        return -1;
    }

    Node* temp = topNode;

    int value = temp->data;

    topNode = topNode->next;

    delete temp;

    count--;

    return value;
}

int top()
{
    if (isEmpty())
    {
```

code 1.cpp | code 2.cpp | code 3.cpp

```
76
77          return topNode->data;
78      }
79
80      int size()
81      {
82          return count;
83      }
84  };
85
86  int main()
87  {
88      Stack s;
89
90      s.push(10);
91      s.push(20);
92      s.push(30);
93
94      cout << "Top element: " << s.top() << endl;
95      cout << "Stack size: " << s.size() << endl;
96
97      cout << "Popped: " << s.pop() << endl;
98      cout << "Popped: " << s.pop() << endl;
99
100     cout << "Top element: " << s.top() << endl;
101     cout << "Stack size: " << s.size() << endl;
102
103     cout << "Is stack empty? ";
104
105     if (s.isEmpty())
106         cout << "Yes" << endl;
107     else
108         cout << "No" << endl;
109
110     return 0;
111 }
```

C:\D local\BSAI\3rd Semester'

```
Top element: 30
Stack size: 3
Popped: 30
Popped: 20
Top element: 10
Stack size: 1
Is stack empty? No

--------------------------------
Process exited after 2.631 seconds with return value 0
Press any key to continue . . .
```

code 1.cpp | code 2.cpp | code 3.cpp

```cpp
#include <iostream>
using namespace std;

class Queue
{
private:

    // Node of Linked List
    struct Node
    {
        int data;
        Node* next;
    };

    Node* front;
    Node* rear;
    int count;

public:

    // 1. Constructor
    Queue()
    {
        front = NULL;
        rear = NULL;
        count = 0;
    }

    // 2. Destructor
    ~Queue()
    {
        while (front != NULL)
        {
            Node* temp = front;
            front = front->next;
            delete temp;
        }

        rear = NULL;
    }
```

code 1.cpp | code 2.cpp | code 3.cpp

```
42      // 3. isEmpty
43      bool isEmpty()
44      {
45          return front == NULL;
46      }
47
48      // 4. push - Add element at REAR
49      void push(int value)
50      {
51          Node* newNode = new Node;
52
53          newNode->data = value;
54          newNode->next = NULL;
55
56          // If queue is empty
57          if (isEmpty())
58          {
59              front = newNode;
60              rear = newNode;
61          }
62          else
63          {
64              rear->next = newNode;
65              rear = newNode;
66          }
67
68          count++;
69      }
70
71      // 5. pop - Remove element from FRONT
72      int pop()
73      {
74          if (isEmpty())
75          {
76              cout << "Queue Underflow!" << endl;
77              return -1;
```

```
        Node* temp = front;

        int value = temp->data;

        front = front->next;

        // If queue becomes empty
        if (front == NULL)
        {
            rear = NULL;
        }

        delete temp;

        count--;

        return value;
    }

    // 6. top - View FRONT element
    int top()
    {
        if (isEmpty())
        {
            cout << "Queue is empty!" << endl;
            return -1;
        }

        return front->data;
    }

    // 7. size
    int size()
    {
        return count;
    }
};

int main()
{
    Queue q;

    // Add elements
    q.push(10);
    q.push(20);
    q.push(30);

    cout << "Front element: " << q.top() << endl;
    cout << "Queue size: " << q.size() << endl;

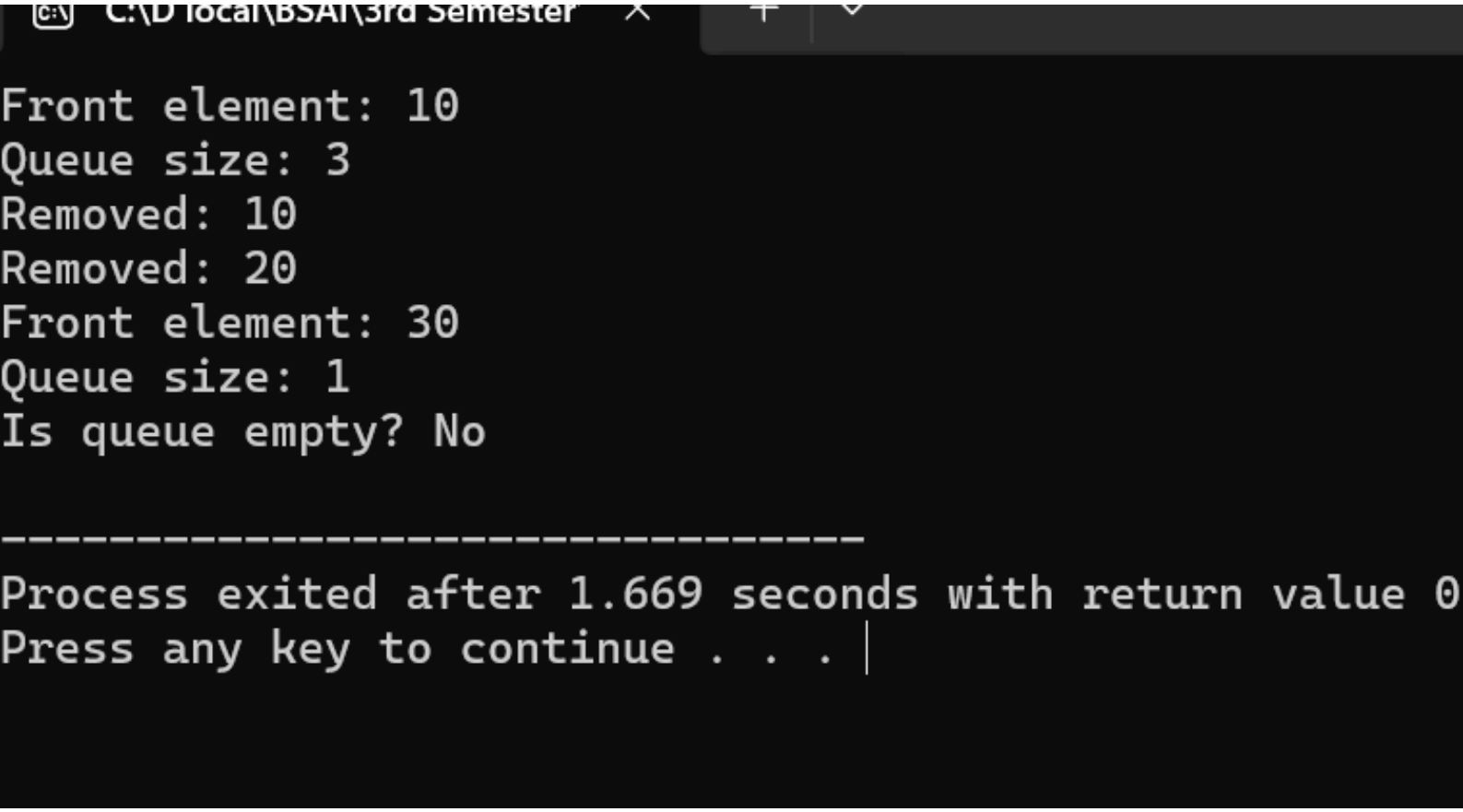
    // Remove elements
    cout << "Removed: " << q.pop() << endl;
    cout << "Removed: " << q.pop() << endl;

    cout << "Front element: " << q.top() << endl;
    cout << "Queue size: " << q.size() << endl;

    cout << "Is queue empty? ";

    if (q.isEmpty())
        cout << "Yes" << endl;
    else
        cout << "No" << endl;

    return 0;
}
```


C:\D local\BSAI\3rd Semester
Front element: 10
Queue size: 3
Removed: 10
Removed: 20
Front element: 30
Queue size: 1
Is queue empty? No
--------------------------------
Process exited after 1.669 seconds with return value 0
Press any key to continue . . .

```cpp
#include <iostream>
#include <string>
using namespace std;

bool isPalindrome(string str)
{
    char stack[100];
    int top = -1;

    // Step 1: Push every character into stack
    for (int i = 0; i < str.length(); i++)
    {
        top++;
        stack[top] = str[i];
    }

    // Step 2: Compare string with stack (reverse order)

    for (int i = 0; i < str.length(); i++)
    {
        if (str[i] != stack[top])
        {
            return false;
        }

        top--;
    }

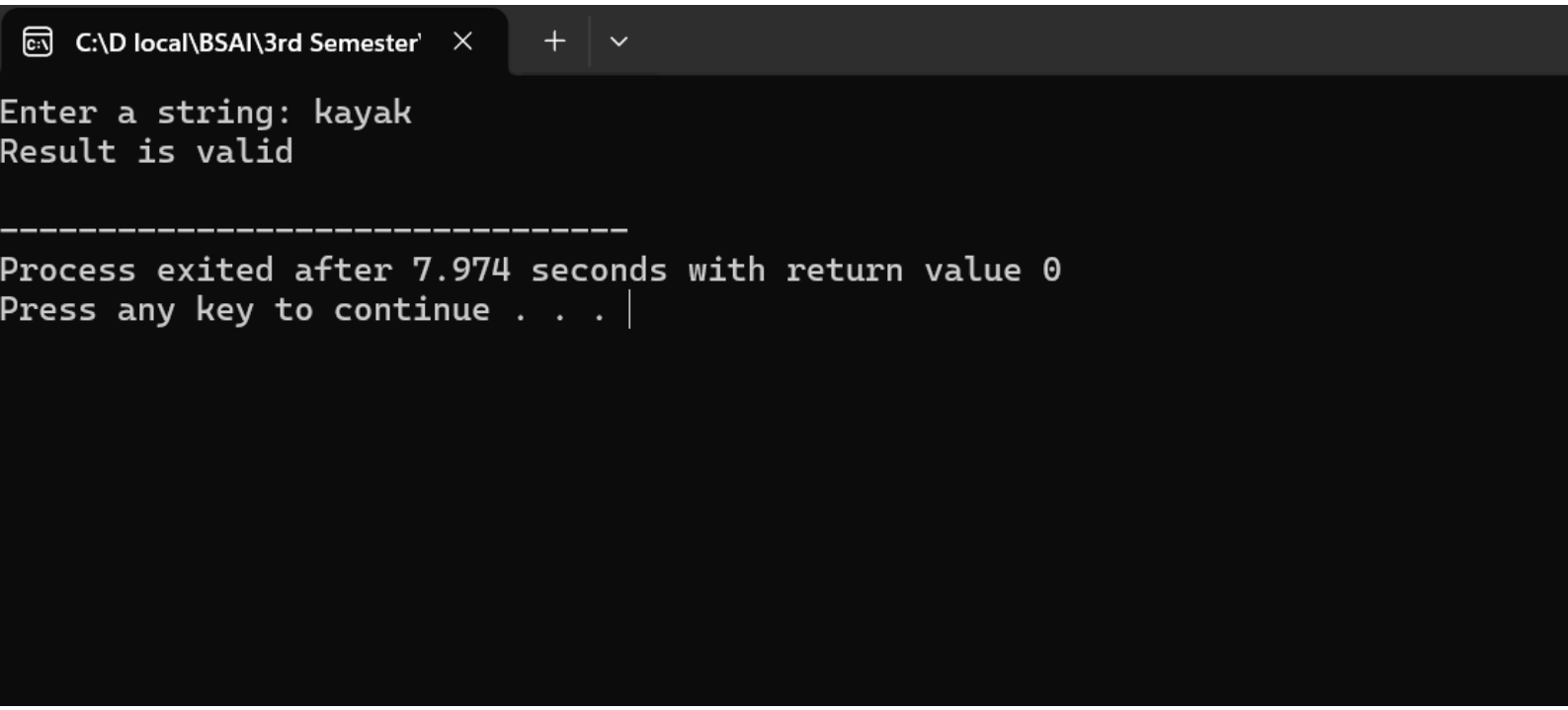
    return true;
}

int main()
{
    string str;

    cout << "Enter a string: ";
    cin >> str;

    if (isPalindrome(str))
        cout << "Result is valid" << endl;
    else
        cout << "Result is invalid" << endl;

    return 0;
}
```


C:\D local\BSAI\3rd Semester'
Enter a string: kayak
Result is valid
--------------------------------
Process exited after 7.974 seconds with return value 0
Press any key to continue . . .